

03 — Logging Configuration: Capturing the Right Events in Production

Table of Contents

1. [Learning Objectives](#)
 2. [Why Logging Matters](#)
 3. [Log Destinations](#)
 4. [Log Line Prefix](#)
 5. [Slow Query Logging](#)
 6. [Checkpoint and WAL Logging](#)
 7. [Lock Wait Logging](#)
 8. [Temporary File Logging](#)
 9. [Autovacuum Logging](#)
 10. [Connection Logging](#)
 11. [Statement-Level Logging](#)
 12. [Complete postgresql.conf Logging Template](#)
 13. [pgBadger for Log Analysis](#)
 14. [Log Rotation and Management](#)
 15. [Structured Logging with csvlog and jsonlog](#)
 16. [Common Mistakes](#)
 17. [Best Practices](#)
 18. [Interview Questions and Answers](#)
 19. [Cross-References](#)
-

Learning Objectives

By the end of this section you will be able to:

- Configure every important logging parameter in `postgresql.conf` with production-appropriate values
 - Explain the trade-offs between logging verbosity and performance overhead
 - Format `log_line_prefix` to include all fields needed for incident analysis
 - Set up CSV and JSON log formats for structured log ingestion
 - Use pgBadger to generate query performance reports from PostgreSQL logs
 - Describe the interaction between log settings and alerting pipelines
-

Why Logging Matters

PostgreSQL statistics views (`pg_stat_*`) show you the *current state* of the database. Logs give you the *history* — what happened at 3:47 AM when the on-call engineer was asleep. A properly configured logging setup answers questions like:

- Which query has been running for 45 minutes?
- Was there a lock wait cascade at the time of the incident?
- Did a checkpoint take unusually long?
- Why did the application start getting errors at a specific time?
- Is autovacuum silently failing on a critical table?

Logs are also the primary input to pgBadger, which aggregates thousands of slow query log entries into ranked performance reports.

Log Destinations

Configured with `log_destination` in `postgresql.conf`.

stderr (default)

Logs are written to PostgreSQL's standard error. This is captured by the system service manager (systemd journal, or redirected to a file via `logging_collector`).

```
log_destination = 'stderr'
logging_collector = on           # enables background log writer process
log_directory = 'log'           # relative to data_directory, or absolute path
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_rotation_age = 1d           # new log file daily
log_rotation_size = 1GB         # or when file exceeds 1GB
log_truncate_on_rotation = off  # don't truncate on time-based rotation
```

csvlog

Logs each entry as a CSV row, suitable for loading into a PostgreSQL table or ingesting into a log aggregator.

```
log_destination = 'stderr,csvlog' # can combine multiple destinations
logging_collector = on
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
```

jsonlog (PostgreSQL 15+)

Logs each entry as a JSON object. Ideal for ingestion into Elasticsearch, Loki, or other JSON-native log stores.

```
log_destination = 'stderr,jsonlog'
logging_collector = on
```

syslog

Routes logs to the OS syslog daemon (rsyslog, syslog-ng). Useful when centralizing all system logs.

```
log_destination = 'syslog'
syslog_facility = 'LOCAL0'
syslog_ident = 'postgres'
```

Log Line Prefix

`log_line_prefix` sets a format string prepended to every log entry. Each `%` escape is replaced with contextual data.

Recommended Production Format

```
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
```

Escape Sequences Reference

Escape	Field	Example Value
%t	Timestamp (no milliseconds)	2025-01-15 14:32:01 UTC
%m	Timestamp with milliseconds	2025-01-15 14:32:01.456 UTC
%n	Epoch timestamp with milliseconds	1736951521.456
%p	Process ID (PID)	12345
%l	Log line number for this session	1
%u	User name	app_user
%d	Database name	production
%a	Application name	myapp
%h	Remote host (IP or hostname)	10.0.1.50
%r	Remote host + port	10.0.1.50(45678)
%b	Backend type	client backend
%c	Session ID (pid.starttime)	5e8f2a1.1
%s	Process start timestamp	2025-01-15 14:00:00 UTC
%v	Virtual transaction ID	5/32
%x	Transaction ID (0 if none)	7890
%q	Quiet — nothing at non-session processes	n/a

Example Log Lines with Recommended Prefix

```
2025-01-15 14:32:01 UTC [12345]: [1-1] user=app,db=orders,app=myapp,client=10.0.1.50
LOG:  duration: 1523.234 ms  statement: SELECT * FROM orders WHERE customer_id = 12345
2025-01-15 14:32:05 UTC [12345]: [2-1] user=app,db=orders,app=myapp,client=10.0.1.50
LOG:  checkpoint starting: time
2025-01-15 14:32:47 UTC [12345]: [3-1] user=app,db=orders,app=myapp,client=10.0.1.50
LOG:  checkpoint complete: wrote 4521 buffers (3.5%); ...
```

Having `user` , `db` , `app` , and `client` on every line allows `grep/pgBadger` to slice the analysis by any of these dimensions.

Slow Query Logging

The most important logging setting for performance analysis.

log_min_duration_statement

Logs every statement that takes longer than the specified duration (in milliseconds). 0 logs all statements; -1 disables.

```
log_min_duration_statement = 1000    # log queries slower than 1 second
```

Production Recommendations:

Environment	Setting	Rationale
Production	1000ms	Low noise, catches problematic queries
Production (stricter SLA)	200ms–500ms	Catch subtler regressions
Development/Staging	0	Log everything for analysis
High-frequency OLTP	100ms	When even 100ms is unacceptable

Note: `log_min_duration_statement` logs the complete normalized query with actual parameter values (unlike `pg_stat_statements` which normalizes them). This is essential for reproducing the exact slow query.

log_min_duration_sample

(PostgreSQL 13+) Sample a percentage of statements below `log_min_duration_statement`. Useful for statistical analysis without logging every fast query.

```
log_min_duration_sample = 100        # sample queries > 100ms
log_statement_sample_rate = 0.01     # log 1% of sampled queries
```

Auto_explain Extension

For queries that exceed a threshold, `auto_explain` logs the execution plan automatically — no manual `EXPLAIN ANALYZE` needed.

```
# In postgresql.conf:
shared_preload_libraries = 'auto_explain'
auto_explain.log_min_duration = 5000    # log plan for queries > 5s
auto_explain.log_analyze = true         # include actual row counts
auto_explain.log_buffers = true         # include buffer usage
auto_explain.log_format = text          # or 'json' for structured
auto_explain.log_nested_statements = true
```

Checkpoint and WAL Logging

log_checkpoints

Logs the start and completion of each checkpoint with statistics.

```
log_checkpoints = on
```

Example log output:

```
LOG:  checkpoint starting: time
LOG:  checkpoint complete: wrote 2847 buffers (2.2%); 0 WAL file(s) added, 1 removed,
2 recycled; write=3.521 s, sync=0.022 s, total=3.589 s; sync files=14, longest=0.012
s, average=0.002 s; distance=24576 kB, estimate=26419 kB
```

Fields to watch:

- `write time`: time spent writing buffers (should be < `checkpoint_completion_target * checkpoint_timeout`)
- `sync time`: fsync time (should be fast on SSD; slow on spinning disk or under I/O pressure)
- `buffers written` : very high values indicate write-heavy workload
- `distance` : WAL distance between checkpoints (compare to `max_wal_size`)

Lock Wait Logging

log_lock_waits

Logs a message whenever a session waits longer than `deadlock_timeout` for a lock. Essential for diagnosing lock contention incidents after the fact.

```
log_lock_waits = on
deadlock_timeout = 1s    # also the threshold for log_lock_waits messages
```

Example log output:

```
LOG:  process 12345 still waiting for ShareLock on transaction 7890 after 1000.234 ms
DETAIL:  Process holding the lock: 12346. Wait queue: 12345.
CONTEXT:  while updating tuple (0,1) in relation "orders"
STATEMENT:  UPDATE orders SET status = 'shipped' WHERE order_id = 99;
```

This gives you: which lock type was waited for, who was holding it, how long the wait was, and the exact SQL. With `log_line_prefix` including `user` , `db` , and `client` , you can trace back to the application component responsible.

Temporary File Logging

log_temp_files

Logs the creation of temporary files used for sort and hash operations that spill to disk (caused by `work_mem` being too small).

```
log_temp_files = 0    # log all temp files regardless of size
# or
log_temp_files = 10240    # log temp files larger than 10MB
```

Example log output:

```
LOG:  temporary file: path "base/pgsql_tmp/pgsql_tmp12345.0", size 104857600
```

Frequent temp file creation indicates `work_mem` needs to be increased for the queries creating them, or those queries need better indexes to avoid large sort operations.

Autovacuum Logging

log_autovacuum_min_duration

Logs autovacuum and autoanalyze runs that take longer than the specified duration.

```
log_autovacuum_min_duration = 0      # log all autovacuum activity
# or
log_autovacuum_min_duration = 250    # log runs > 250ms
```

Example log output:

```
LOG:  automatic vacuum of table "production.public.orders": index scans: 1
      pages: 0 removed, 12450 remain, 0 skipped due to pins, 0 skipped frozen
      tuples: 45230 removed, 890234 remain, 0 are dead but not yet removable, oldest
xmin: 7890123
      buffer usage: 18234 hits, 1290 misses, 892 dirtied
      avg read rate: 2.340 MB/s, avg write rate: 1.621 MB/s
      system usage: CPU: user: 1.23 s, system: 0.12 s, elapsed: 23.45 s
```

This tells you: how many tuples were removed, how many remain dead but cannot yet be removed (blocked by old transactions), buffer cache hit rate during vacuum, and CPU/elapsed time. The "oldest xmin" field is critical — if it's not advancing, a long-running transaction is preventing vacuum from cleaning up.

Connection Logging

log_connections and log_disconnections

```
log_connections = on
log_disconnections = on
```

When to enable:

- Security auditing requirements
- Debugging connection pool behavior
- Investigating connection exhaustion incidents

When to disable:

- High-frequency connection creation (e.g., without a connection pool), where connection logs would dominate the log volume
- Applications that create thousands of short-lived connections per minute

Example output:

```
LOG:  connection received: host=10.0.1.50 port=45678
LOG:  connection authorized: user=app database=orders application_name=myapp
LOG:  disconnection: session time: 0:00:02.456 user=app database=orders host=10.0.1.50
port=45678
```

Statement-Level Logging

log_statement

Controls which SQL statements are always logged (regardless of duration).

Value	What Is Logged	Use Case
none	Nothing	Production default
ddl	CREATE, DROP, ALTER, GRANT, REVOKE	Audit DDL changes
mod	DDL + INSERT, UPDATE, DELETE, TRUNCATE	Audit data modifications
all	Every statement	Development/debugging only

```
log_statement = 'ddl'    # recommended for production audit trail
```

Warning: `log_statement = 'all'` on a busy production database will generate enormous log volumes and can itself become a performance bottleneck.

Complete postgresql.conf Logging Template

```
# =====
# LOGGING CONFIGURATION – Production Template
# =====

# --- Destination ---
log_destination = 'stderr, csvlog'    # stderr for systemd; csvlog for analytics
logging_collector = on
log_directory = '/var/log/postgresql'
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
log_file_mode = 0640
log_rotation_age = 1d
log_rotation_size = 512MB
log_truncate_on_rotation = off

# --- Line Prefix ---
# Includes: timestamp with ms, pid, line#, user, db, app, client IP
log_line_prefix = '%m [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '

# --- Slow Query Logging ---
log_min_duration_statement = 1000    # ms; log queries > 1 second
# log_min_duration_sample = 100      # PG13+: also sample queries > 100ms
# log_statement_sample_rate = 0.01   # PG13+: at 1% sample rate

# --- Checkpoint Logging ---
log_checkpoints = on

# --- Lock Wait Logging ---
log_lock_waits = on
```

```
deadlock_timeout = 1s

# --- Temp File Logging ---
log_temp_files = 10240 # log temp files > 10MB

# --- Autovacuum Logging ---
log_autovacuum_min_duration = 250 # log autovacuum runs > 250ms

# --- Connection Logging ---
log_connections = on # enable if connection pooler is in use
log_disconnections = on

# --- Statement Logging ---
log_statement = 'ddl' # always log DDL for audit trail

# --- Error Verbosity ---
log_error_verbosity = default # default, terse, or verbose

# --- Log Level ---
log_min_messages = warning # INFO, NOTICE, WARNING, ERROR, LOG, FATAL, PANIC
log_min_error_statement = error # log the SQL for messages at this level or above

# --- Duration Logging ---
log_duration = off # off = don't log duration of every statement
# (use log_min_duration_statement instead)

# --- Timezone ---
log_timezone = 'UTC' # ALWAYS use UTC in production

# --- Query Plan Logging (via auto_explain) ---
# shared_preload_libraries = 'pg_stat_statements,auto_explain'
# auto_explain.log_min_duration = 5000
# auto_explain.log_analyze = true
# auto_explain.log_buffers = true
```

pgBadger for Log Analysis

pgBadger is an open-source PostgreSQL log analyzer written in Perl. It parses PostgreSQL logs and generates HTML reports showing:

- Top slow queries ranked by total time, average time, and call count
- Lock wait analysis
- Connection statistics
- Error/warning frequency
- Temporary file usage
- Checkpoint statistics
- Autovacuum activity

Installation


```
# Debian/Ubuntu
apt-get install pgbadger

# RHEL/CentOS
yum install pgbadger

# From source
git clone https://github.com/darold/pgbadger.git
cd pgbadger && perl Makefile.PL && make && make install
```

Basic Usage

```
# Generate HTML report from a single log file
pgbadger /var/log/postgresql/postgresql-2025-01-15.log -o /tmp/report.html

# Parse CSV log format (faster)
pgbadger /var/log/postgresql/postgresql-2025-01-15.csv \
  --format csv \
  -o /var/www/pgbadger/report.html

# Parse multiple files (e.g., a week)
pgbadger /var/log/postgresql/postgresql-2025-01-*.log -o /tmp/weekly.html

# Incremental processing (for daily reports)
pgbadger --incremental --outdir /var/www/pgbadger/ \
  /var/log/postgresql/postgresql-*.log
```

Required Log Settings for pgBadger

```
# These are REQUIRED for pgBadger to work correctly:
log_min_duration_statement = 0      # or any value; needed for slow query report
log_line_prefix = '%t [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h '
log_checkpoints = on
log_connections = on
log_disconnections = on
log_lock_waits = on
log_temp_files = 0
log_autovacuum_min_duration = 0

# pgBadger also benefits from:
lc_messages = 'en_US.UTF-8' # pgBadger expects English log messages
```

Daily Automated pgBadger Report (cron)

```
#!/bin/bash
# /etc/cron.daily/pgbadger-report
LOGDIR=/var/log/postgresql
OUTDIR=/var/www/html/pgbadger
```

```
YESTERDAY=$(date -d yesterday +%Y-%m-%d)

pgbadger \
  --format stderr \
  --prefix '%m [%p]: [%l-1] user=%u,db=%d,app=%a,client=%h ' \
  --outfile "${OUTDIR}/report-${YESTERDAY}.html" \
  "${LOGDIR}/postgresql-${YESTERDAY}*.log"
```

Key pgBadger Report Sections

Section	What It Shows	Action
Queries by total duration	Queries spending the most cumulative time	EXPLAIN ANALYZE, add indexes
Queries by mean duration	Consistently slow queries	Plan regression investigation
Queries by call count	Most frequent queries	Cache or optimize hot paths
Lock waits	Lock contention events	Transaction ordering, timeout tuning
Temp files	Queries spilling to disk	Increase work_mem, add indexes
Checkpoints	Checkpoint timing and frequency	Tune max_wal_size, checkpoint_timeout
Autovacuum	Tables vacuumed, time taken	Tune autovacuum parameters
Connections	Connections per user/db/app	Connection pool sizing

Log Rotation and Management

Using logging_collector

PostgreSQL's built-in logging_collector process rotates logs based on age and size:

```
log_rotation_age = 1d          # rotate daily
log_rotation_size = 512MB      # or when file > 512MB
log_truncate_on_rotation = off
log_filename = 'postgresql-%Y-%m-%d_%H%M%S.log'
```

Using logrotate (Linux)

```
# /etc/logrotate.d/postgresql
/var/log/postgresql/*.log {
    daily
    rotate 14
    compress
    delaycompress
    missingok
```

```
    notifempty
    sharedscripts
    postrotate
        /usr/bin/pg_ctlcluster 15 main reload > /dev/null 2>&1 || true
    endscript
}
```

Retention Policy

Log Type	Recommended Retention
Slow query logs	30 days (for trend analysis)
Error/warning logs	90 days (for compliance)
DDL audit logs	1 year
pgBadger reports	90 days
Connection logs	7-14 days (high volume)

Structured Logging with csvlog and jsonlog

CSV Log Schema

The CSV log has 23 fields. To load into PostgreSQL for analysis:

```
-- Create a table matching the CSV log format
CREATE TABLE postgres_log (
    log_time          TIMESTAMP WITH TIME ZONE,
    user_name         TEXT,
    database_name     TEXT,
    process_id        INTEGER,
    connection_from   TEXT,
    session_id        TEXT,
    session_line_num  BIGINT,
    command_tag       TEXT,
    session_start_time TIMESTAMP WITH TIME ZONE,
    virtual_transaction_id TEXT,
    transaction_id    BIGINT,
    error_severity    TEXT,
    sql_state_code    TEXT,
    message           TEXT,
    detail            TEXT,
    hint             TEXT,
    internal_query    TEXT,
    internal_query_pos INTEGER,
    context           TEXT,
    query            TEXT,
    query_pos        INTEGER,
    location          TEXT,
    application_name  TEXT,
```

```

        backend_type          TEXT,    -- added in PG13
        leader_pid            INTEGER, -- added in PG14
        query_id               BIGINT   -- added in PG14
    );

-- Load the CSV log
COPY postgres_log FROM '/var/log/postgresql/postgresql-2025-01-15.csv' WITH CSV;

-- Query top slow queries from the log
SELECT
    message,
    COUNT(*) AS occurrences,
    MAX(SUBSTRING(message FROM 'duration: (\d+\.\d+)')::FLOAT) AS max_ms
FROM postgres_log
WHERE message LIKE 'duration:%'
GROUP BY message
ORDER BY max_ms DESC
LIMIT 10;

```

Common Mistakes

1. **Setting `log_min_duration_statement = 0` in production** without realizing the log volume implications. On a database handling 10,000 TPS, this generates millions of log entries per hour, filling disks and masking important messages.
2. **Using `log_timezone = 'localtime'` instead of UTC.** When correlating logs with application logs or across time zones, local time with DST transitions creates ambiguous timestamps at the clock change.
3. **Not including `%a (application_name)` in `log_line_prefix`.** Without this, you cannot tell which microservice or application component generated a slow query.
4. **Forgetting `log_autovacuum_min_duration = 0` when debugging autovacuum.** By default, this is often -1 (no logging), so autovacuum runs silently even when they are struggling.
5. **Placing log files on the same disk as the data directory.** Heavy logging under I/O pressure can slow down WAL writes. Log to a separate filesystem.
6. **Not setting `lc_messages = 'en_US.UTF-8'`.** pgBadger parses English log messages. Non-English locales produce log entries that pgBadger cannot interpret.
7. **Leaving `log_connections = on` on systems without a connection pool.** Applications that open/close connections for every request will generate thousands of connection log entries per second.

Best Practices

- **Always set `log_timezone = 'UTC'`** to avoid time zone confusion during incident investigations.
- **Enable `log_lock_waits = on` and `log_checkpoints = on`** with zero performance overhead — these generate events only when those conditions occur.
- **Enable `log_statement = 'ddl'`** as a minimum for an audit trail of schema changes.

- Use `csvlog` or `jsonlog` alongside `stderr` for machine-readable log ingestion.
 - Deploy pgBadger as a nightly report and review it every morning. Many production degradations show up in pgBadger before users report them.
 - Test your logging configuration in staging before applying to production: run a load test and verify that log volume is manageable.
 - Set `log_min_duration_statement` per role using `ALTER ROLE myapp SET log_min_duration_statement = 500;` to avoid lowering the threshold globally.
 - Include `query_id` in your log prefix (PG14+) to correlate log entries with `pg_stat_statements.queryid`.
-

Interview Questions and Answers

Q1: What is `log_min_duration_statement` and what is a reasonable value for a production OLTP system?

A: `log_min_duration_statement` is a GUC (Grand Unified Configuration parameter) that specifies a duration threshold in milliseconds. Any SQL statement that takes longer than this threshold is logged with its text and execution time. For OLTP systems with sub-100ms SLAs, a common production value is 1000ms (1 second) as a safety net for catching rogue queries, with a stricter value of 200–500ms if the team wants earlier visibility into degrading queries. Setting it to 0 (log everything) is appropriate for development but causes excessive log volume in production.

Q2: What is the purpose of `log_line_prefix` and what fields should always be included?

A: `log_line_prefix` prepends metadata to every log entry. The essential fields for production are: `%m` (timestamp with milliseconds), `%p` (PID), `%u` (username), `%d` (database), `%a` (application name), and `%h` (client host). These fields allow you to correlate log entries with a specific application, client connection, and transaction. Without `%a`, you cannot distinguish which application generated a particular slow query. Without `%p`, you cannot correlate multi-line log entries from the same session.

Q3: Why should `log_lock_waits = on` always be enabled in production?

A: Lock wait events are one of the most common causes of production slowness and are invisible without logging. When a session waits longer than `deadlock_timeout` for a lock, PostgreSQL logs the blocking PID, the SQL statement being blocked, the lock type, and the wait duration. This information is irreplaceable during post-incident analysis — the lock wait may have resolved by the time the engineer investigates, and there would be no trace without logging. The overhead is negligible because the log entry is written only when the wait actually occurs.

Q4: How does `auto_explain` differ from `log_min_duration_statement` ?

A: `log_min_duration_statement` logs the SQL text and execution time of slow queries. `auto_explain` logs the full execution plan (from `EXPLAIN ANALYZE`) for slow queries. The plan includes actual row counts, execution times per node, buffer usage, and join strategies — all the information you would normally get by running `EXPLAIN ANALYZE` manually. The combination is powerful: `log_min_duration_statement` catches slow queries; `auto_explain` tells you exactly *why* they were slow without requiring you to reproduce the query.

Q5: What log settings are required for pgBadger to produce useful reports?

A: pgBadger requires at minimum: `log_min_duration_statement` set to some value (it parses duration: lines), `log_line_prefix` in a format pgBadger understands, and `lc_messages = 'en_US.UTF-8'` (pgBadger parses English log message text). For comprehensive reports, also enable: `log_checkpoints`,

`log_connections` , `log_disconnections` , `log_lock_waits` , `log_temp_files` , and `log_autovacuum_min_duration` . pgBadger supports stderr, csvlog, and syslog formats.

Q6: What is the difference between `log_min_messages` and `log_min_error_statement` ?

A: `log_min_messages` controls the minimum severity level of messages that are written to the log at all (DEBUG5 through PANIC). The default is `WARNING` , meaning DEBUG, INFO, and NOTICE messages are suppressed. `log_min_error_statement` controls the minimum severity level at which the SQL statement that caused the message is also logged. For example, with `log_min_error_statement = error` , whenever an ERROR occurs, both the error message and the SQL that triggered it are logged — useful for debugging application errors.

Q7: How would you set up logging to capture only slow queries from a specific application without affecting other applications?

A: Use `ALTER ROLE` or `ALTER DATABASE` to set GUCs per-connection:

```
-- For a specific role:  
ALTER ROLE my_app_user SET log_min_duration_statement = 200;  
-- For a specific database:  
ALTER DATABASE mydb SET log_min_duration_statement = 500;  
-- Reset to global:  
ALTER ROLE my_app_user RESET log_min_duration_statement;
```

These role/database-level settings override the `postgresql.conf` value for connections matching that role or database. This is useful when you want verbose logging for a specific service under investigation without flooding the logs with entries from other applications.

Q8: What does `log_temp_files = 0` do and what does seeing frequent temp file log entries indicate?

A: `log_temp_files = 0` logs the creation of every temporary file (regardless of size) used by sort and hash operations that spill to disk. Frequent temp file entries indicate that `work_mem` is too small for the queries generating them — the sort or hash operation exceeds the memory budget and falls back to disk. The fix is to increase `work_mem` globally or per-query (`SET work_mem = '256MB'` before the query). However, be careful: `work_mem` is allocated per sort/hash *per parallel worker*, so a query with 10 parallel workers and 5 sorts can use 50x `work_mem` .

Cross-References

- `01_monitoring_overview.md` — monitoring layer overview
- `04_observability_stack.md` — log shipping to centralized stores (Loki, Elasticsearch)
- `06_maintenance_procedures.md` — using autovacuum logs to tune autovacuum parameters
- `07_incident_response.md` — log queries used in runbooks
- `08_performance_tuning_config.md` — `auto_explain` and logging parameters in the full config template
- `09_troubleshooting_guide.md` — using logs as the starting point for diagnosis